RAJIV GANDHI INSTITUTE OF PETROLEUM TECHNOLOGY, JAIS, AMETHI

Department of Computer Science and Engineering



B.Tech. 2nd Year

Object Oriented Methodologies (CS224)

By

Dr. Kalka Dubey

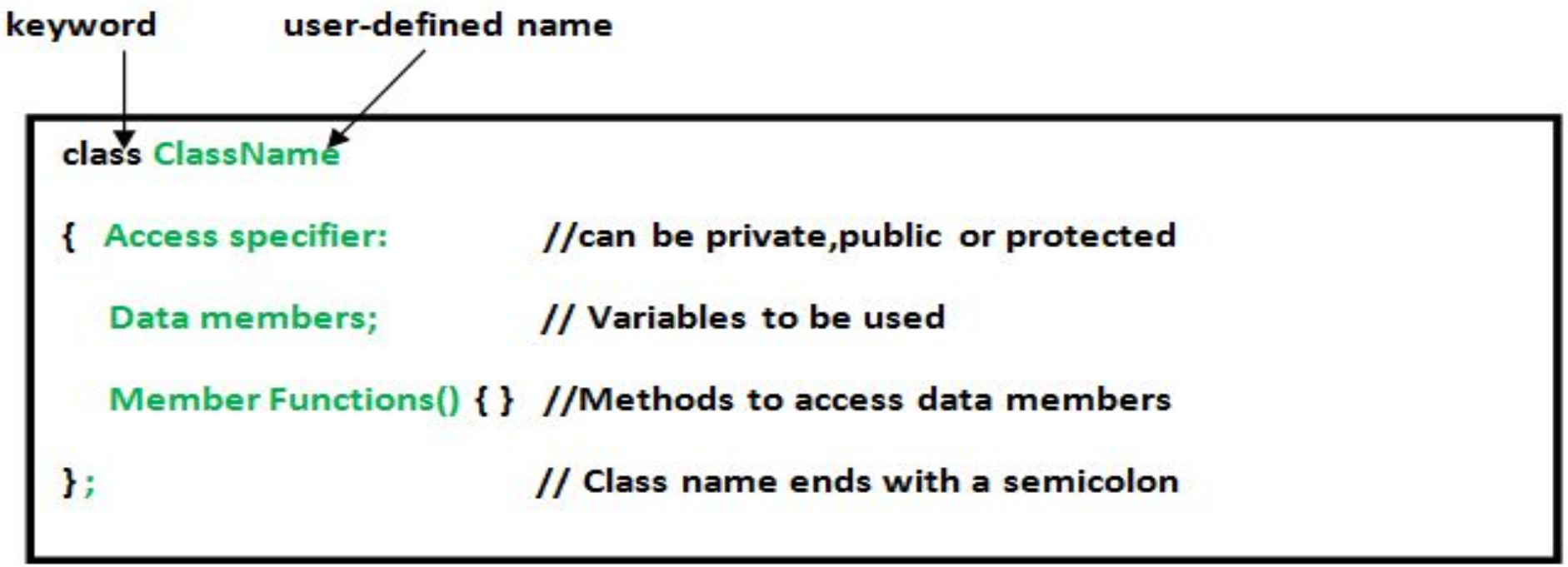
UNIT-II (Classes & Objects)

- *Objects*
- *Classes*
- *Specification of Class*
- *Visibility Modes: Private, Public, Protected*
- *Defining Member Functions*
- *Creating of Objects*
- *Characteristics of Object*
- *Static Data Member*
- *Static Member Function, Methods*
- *Constructors*
- *Types of Constructors*
- *Rules of Constructor Definition and Usage*
- *Destructor*

Class

- The most remarkable feature of C + + is a class.
- The class binds together data and methods which work on data.
- The class is an abstract data type (ADI) so the creation of a class simply creates a template.
- The body of the class is contained within a set of braces, { } ;
- A class definition begins with the keyword class.
- The data and functions within a class are called members of the class.
- The declaration of a class alone does not serve any purpose.

Class


keyword
user-defined name
class ClassName
{ Access specifier: //can be private,public or protected
Data members; // Variables to be used
Member Functions() { } //Methods to access data members
}; // Class name ends with a semicolon

Access Specifier

- There are three access modifiers.

- Public
- Private
- Protected.

- Specify the access level of the members of the class. (the default is private)

Access Specifier

- If the class member is declared as public then it can be accessed everywhere.

- If the class members are declared as protected then it can be accessed only within the class itself and by inheriting and parent classes.

- If the class members are declared as private then it may only be accessed by the class that defines the member.

Member of a Class

- Data Member
- Member Function

Data Member

- The variables which are declared in any class by using any fundamental data types (like int, char, float etc) or derived data type (like class, structure, pointer etc.) are known as Data Members.
- Accessing a data member depends solely on the access control of that data member.
- If its public, then the data member can be easily accessed using the direct member access (.) operator with the object of that class.
- If, the data member is defined as private or protected, then we cannot access the data variables directly.
- Then we will have to create special public member functions to access, use or initialize the private and protected data members. These member functions are also called Accessors and Mutator methods or getter and setter functions.

Defining a Member Function

- There are 2 ways to define a member function:

 Inside class definition

 Outside class definition

- To define a member function outside the class definition we have to use the scope resolution :: operator along with class name and function name.

- All the member functions defined inside the class definition are by default inline.

- To make any non-class function inline by using keyword inline with them.

Object

- An Object is an instance of a Class.
- Declaring a variable of a class type creates an object.
- When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.
- The creation of object creates memory space for the object which depends upon the size of the data members of the class.
- The objects call member functions of the class using operator dot (.) operator. Which is known as period or membership operator.

Object Creation

- In C++, an object is created from a class.
- To create an object of an class, specify the class name, followed by the object name.

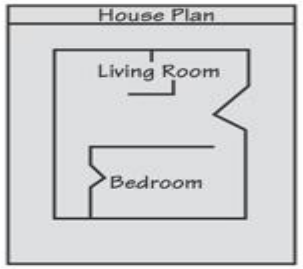
Syntax:

```
ClassName ObjectName;
```

Classes and Objects

- A Class is like a blueprint and objects are like houses built from the blueprint

Blueprint that describes a house.



Instances of the house described by the blueprint.



C++ Class definitions

- A class definition starts with the keyword *class* followed by the class name; and the class body, enclosed by a pair of curly braces. A class definition must be followed either by a semicolon or a list of declarations.

```
class Box
{
  public:
     double length;   // Length of a box
     double breadth;  // Breadth of a box
     double height;   // Height of a box
};
```

Define C++ Objects

- A class provides the blueprints for objects, so basically an object is created from a class. We declare objects of a class with exactly the same sort of declaration that we declare variables of basic types.

```
Box Box1;        // Declare Box1 of type Box
Box Box2;        // Declare Box2 of type Box
```

Accessing data members

- The public data members of objects of a class can be accessed using the direct member access operator (.)

```
// box 1 specification
   Box1.height = 5.0;
   Box1.length = 6.0;
   Box1.breadth = 7.0;

   // box 2 specification
   Box2.height = 10.0;
   Box2.length = 12.0;
   Box2.breadth = 13.0;
```

Defining member functions

- A member function of a class is a function that has its definition within the class or separately using scope resolution operator, ::

```
double Box::getVolume(void)
{
   return length * breadth * height;
}
```

Calling a member functions

- A member function will be called using a dot operator (.)

```
Box myBox;          // Create an object
myBox.getVolume();
```

Example 1

```
class Box
{
  public:
    double length;
    double breadth;
    double height;

    double getVolume(void);
    void setLength( double len );
    void setBreadth( double bre );
    void setHeight( double hei );
};
```

```
// Member functions definitions
double Box::getVolume(void)
{
   return length * breadth * height;
}

void Box::setLength( double len )
{
   length = len;
}

void Box::setBreadth( double bre )
{
   breadth = bre;
}

void Box::setHeight( double hei )
{
   height = hei;
}
```

```
// Main function for the program
int main( )
{
  Box Box1;
  double volume = 0.0;

  // box 1 specification
  Box1.setLength(6.0);
  Box1.setBreadth(7.0);
  Box1.setHeight(5.0);

  // volume of box 1
  volume = Box1.getVolume();
  cout << volume <<endl;

  return 0;
}
```

Example 2

```
class Box
{
  public:
    double length;
    double breadth;
    double height;

    double getVolume(void);
    void setLength( double len );
    void setBreadth( double bre );
    void setHeight( double hei );
};
```

```
// Member functions definitions
double Box::getVolume(void)
{
   return length * breadth * height;
}

void Box::setLength( double len )
{
   length = len;
}

void Box::setBreadth( double bre )
{
   breadth = bre;
}

void Box::setHeight( double hei )
{
   height = hei;
   getVolume();
}
```

```
// Main function for the program
int main( )
{
  Box Box1;
  double volume = 0.0;

  // box 1 specification
  Box1.setLength(6.0);
  Box1.setBreadth(7.0);
  Box1.setHeight(5.0);

  return 0;
}
```

Nesting of Member Functions

C++ access modifiers

- The access restriction to the class members is specified by the labeled public, private, and protected sections within the class body.

```
class Base {
   public:
  // public members go here

  protected:
 // protected members go here

  private:
  // private members go here
};
```

- **Public : A public member is accessible from anywhere outside the class but within a program**

```
class Line
{
  public:
     double length;
     void setLength( double len );
     double getLength( void );
};

// Member functions definitions
double Line::getLength(void)
{
   return length ;
}
void Line::setLength( double len )
{
   length = len;
} };
```

```
int main( )
{
  Line line;

  // set line length
  line.setLength(6.0);
  cout << "Length of line : " << line.getLength() <<endl;

  // set line length without member function
  line.length = 10.0; // OK: because length is public
  cout << "Length of line : " << line.length <<endl;
  return 0;
}
```

- Private : A private member variable or function cannot be accessed, or even viewed from outside the class. Only the class and friend functions can access private members.

```
class Line
{
  private:
     double length;
  public:
     void setLength( double l );
};

void Box::setLength( double l )
{
   length= l;
}
```

```
int main( )
{
  Line line;
  line.length=20.0; // wrong
  line.setLength(20.0); // Use member function to set it.
  return 0;
}
```

- **Protected: A protected member variable or function is very similar to a private member but it provided one additional benefit that they can be accessed in child classes which are called derived classes.**

```
class Box
{
   protected:
      double width;
};

class SmallBox:Box //derived class.
{
   public:
      void setSmallWidth( double wid );
      double getSmallWidth( void );
};

// Member functions of child class
double SmallBox::getSmallWidth(void)
{
    return width ;
}

void SmallBox::setSmallWidth( double wid )
{
    width = wid;
}

int main( )
{
   SmallBox box;

   // set box width using member function
   box.setSmallWidth(5.0);
   cout << "Width of box : "<< box.getSmallWidth() <<
endl;

   return 0;
}
```

- **Write a program using class to declare member variable and functions private, public and protected section and make an attempt to access them using object.**

```
#include<iostream.h>
#include<conio.h>
class sample
{
private:
int num;
void show1()
{
cout<<“Inside the private section”;
cout<<“\nEnter a number:”;
cin>>num;
cout<<“Number is:”<<num;
}
public:
int num1;
void show()
{
show1();
cout<<“\n\nInside the public section”;
cout<<“\nEnter a number:”;
cin>>num1;
cout<<“Number is:”<<num1;
show2();
}
protected:
int num2;
void show2()
{
cout<<“\n\nInside the protected section”;
cout<<“\nEnter a number:”;
cin>>num2;
cout<<“Number is:”<<num2;
}
};
int main()
{
sample s;
s.show();
return 0;
}
```

OUTPUT

```
Inside the private section
Enter a number:4
Number is:4
Inside the public section
Enter a number:5
Number is:5
Inside the protected section
Enter a number:6
Number is:6
```

- **Write a program to calculate simple interest. Hide the data elements of the class using private keyword.**

```
#include<iostream.h>
#include<conio.h>
class interest
{
private :
float p_amount; // principle amount
float rate; // rate of interest
float period; // numbers of years
float interest;
float t_amount; // total amount
public :
void in()
{
cout<<" Principle Amount : "; cin>>p_amount;
cout<<" Rate of Interest : "; cin>>rate;
cout<<" Number of years : "; cin>>period;
interest=(p_amount*period*rate)/100;
t_amount=interest+p_amount;
}
void show()
{
cout<<"\n Principle Amount : "<<p_amount;
cout<<"\n Rate of Interest : "<<rate;
cout<<"\n Number of years : "<<period;
cout<<"\n Interest : "<<interest;
cout<<"\n Total Amount : "<<t_amount;
}
};
int main()
{
interest r;
r.in();
r.show();
return 0;
}
```

OUTPUT

Principle Amount : 5000
Rate of Interest : 2
Number of years : 3
Principle Amount : 5000
Rate of Interest : 2
Number of years : 3
Interest : 300
Total Amount : 5300

- **An electricity board charges the following rates to domestic users to discourage large consumption of energy:**

  - **For the first 100 units ₹ 7/- per unit**
  - **Beyond 100 units ₹7.5/- per unit**

- **If the total cost of the electricity consumption is more than Rs.250.00 then an additional surcharge of 15% is added to the difference else 10% is added to the difference. Define a class Electricity in which all the data members required are private natures and create a member function count_bill () to compute the total cost of electricity consumption units. The total consumption of the electricity unit by the customer is 253.**

- **Write a C++ program to calculate the above mention requirement along with the use of appropriate data if needed.**

Arrays within the class

- Arrays can be used as member variables in a class.

```
class array
{
int regno[10];
}
```

ARRAYS OF OBJECTS

```
class Details
{
private:
int salary;
float roll;
public:
void getname( )
{
cout << "\n Enter the Salary:";
cin >> salary;
cout << "\n Enter the roll:";
cin >> roll;
}
void putname( )
{
cout << "Employees" << salary <<
"and roll is" << roll << '\n';
}
};
```

```
void main()
{
Details det[50];
int n=0;
char ans;
do
{
cout << "Enter the Employee Number::" << n+1;
det[n++].getname();
cout << "Enter another (y/n)?: " ;
cin >> ans;
}
while ( ans != 'n' );
for (int j=0; j<n; j++)
{
cout << "\nEmployee Number is:: " << j+1;
det[j].putname( );
}
}
```

STRUCTURE AS A MEMBER OF CLASS

```
#include <iostream.h>
#include <string.h>
#include <conio.h>
class person
{
char name[15];
int age;
float sal;
public :
struct
{
char hno[15];
char street[15];
char city[15];
char state[15];
}addr;
void input(char s[ ], int a, float f)
{
strcpy(name, s);
age=a;
sal=f;
}
void main( )
{
person p;
clrscr( );
p.input("HARI",24,15000);
strcpy(p.addr.hno,"s17/127");
strcpy(p.addr.street,"23");
strcpy(p.addr.city,"Varanasi");
strcpy(p.addr.state,"UP");
cout<<"\n\t PERSON DETAILS\n\n";
p.show( );
getch( );
}
```

STRUCTURE AS A MEMBER OF CLASS VER

OUTPUT :

PERSON DETAILS
Name=Vijay
Age=24
Salary=15000

USE OF ENUM AS CLASS MEMBER

```
#include <iostream.h>
#include <conio.h>
class demo
{
public :
enum
{
RED,
GREEN,
BLUE,
YELLOW,
WHITE
};
};
void main( )
{
clrscr( );
cout<<demo : :RED<<endl;
cout<<demo : :GREEN<<endl;
cout<<demo : :BLUE<<endl;
cout<<demo : :YELLOW<<endl;
cout<<demo : :WHITE<<endl;
getch( );
}
```

USE OF ENUM AS CLASS MEMBER

OUTPUT :

0
1
2
3
4

PASSING AND RETURNING OBJECT

- Similar to returning and passing arguments to function of basic type like int, char, double, float, char* etc.
- we can pass objects of class to functions and even return objects from functions.
- To pass object we have simply write in the declaration of function, the base name whose objects we will be passing.
- Similarly for returning an object we will have to write class name as return type.
- For example for a function show which takes an object of demo class type and return an object of demo class.

PASSING AND RETURNING OBJECT Example 1

```
#include <iostream.h>
#include <conio.h>
class demo
{
int num;
public :
void input(int x)
{
num=x;
}
void copy(demo);
void show( )
{
cout<<“num=”<<num<<endl;
}
};
void demo : :copy (demo d)
{
num=d.num;
}
void main( )
{
demo d1,d2;
clrscr( );
d1.input(20);
d2.copy(d1);
cout<<“Object d1\n”<<endl;
d1.show( );
cout<<“Object d2\n”;
d2.show( );
getch( );
}
```

PASSING AND RETURNING OBJECT Example 1

OUTPUT :

```
Object d1
num=20
Object d2
num=20
```

PASSING AND RETURNING OBJECT Example 2

```
#include <iostream.h>
#include <conio.h>
#include <string.h>
class employee
{
float sal;
char name[10];
public :
void input(float s, char n[ ])
{
sal=s;
strcpy(name,n);
}
void comp_sal(employee temp);
};
void employee : : comp_sal(employee temp) // object as argument
{
if(temp.sal>sal)
cout<<temp.name<<“ ‘s salary is higher “
<<“ than “ <<name<<“ ‘s salary”<<endl;
else
cout<<name<<“‘s salary is higher “
<<“ than “<<temp.name<<“ ‘s salary\n”;
}
void main( )
{
employee e1, e2;
e1.input(10000.275,”Hari”);
e2.input(11000.0,”Ravi”);
e1.comp_sal(e2);
getch( );
}
```

PASSING AND RETURNING OBJECT Example 2

OUTPUT :

Ravi ‘s salary is higher than Hari ‘s salary

RETURING OBJECT AS ARGUMENT Example 1

```
#include <iostream.h>
#include <conio.h>
#include <string.h>
class demo
{
int num;
public :
void input(int x)
{
num = x;
}
demo copy( );
void show( )
{
cout<<“num=”<<num<<endl;
}
};
demo demo : :copy( )
{
demo temp;
temp.num = num;
return temp;
}
```

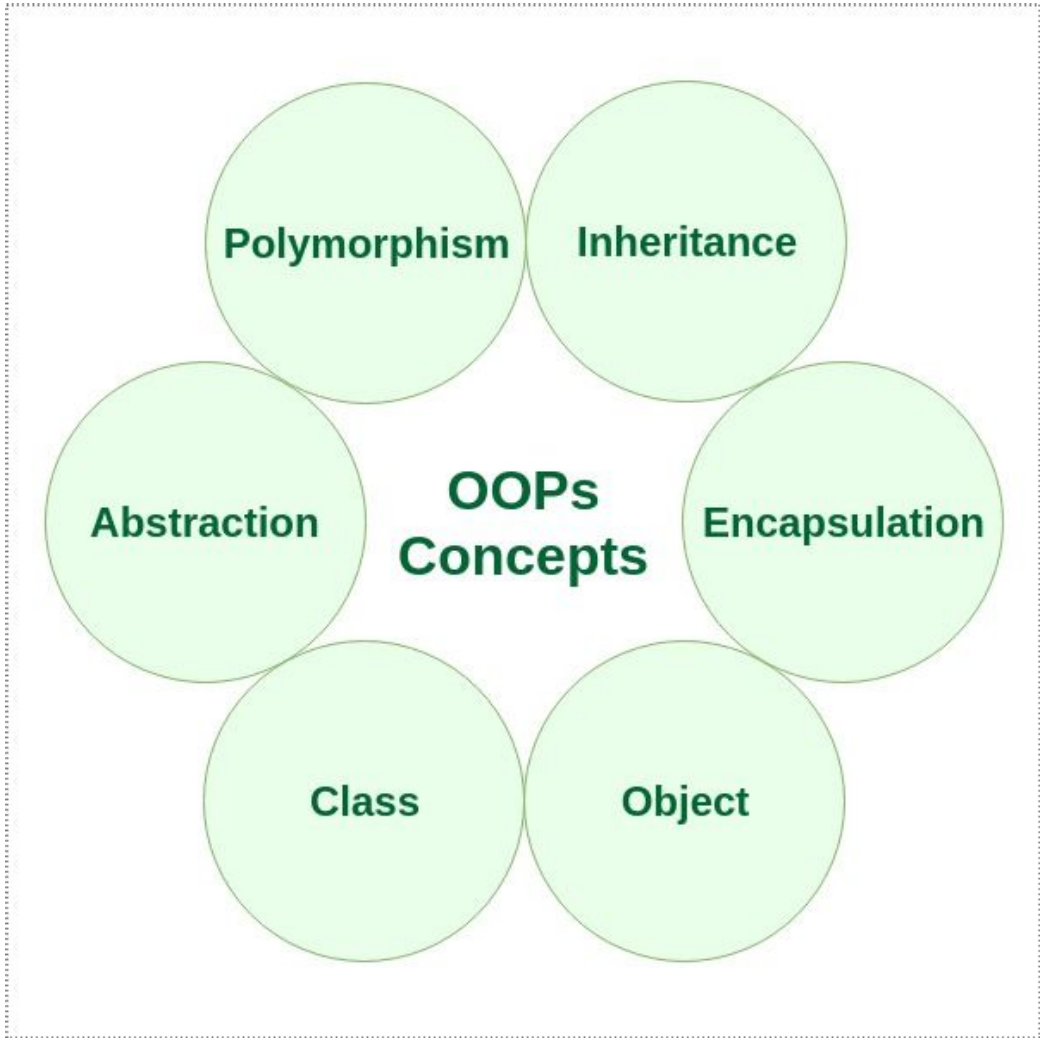
```
void main( )
{
clrscr( );
demo d1, d2;
d1.input(20);
d2=d1.copy( );
cout<<“Object d1\n”;
d1.show( );
cout<<“Object d2\n”;
d2.show( );
}
```

PASSING AND RETURNING OBJECT Example 2

OUTPUT :

Object d1
num=20
Object d2
num=20

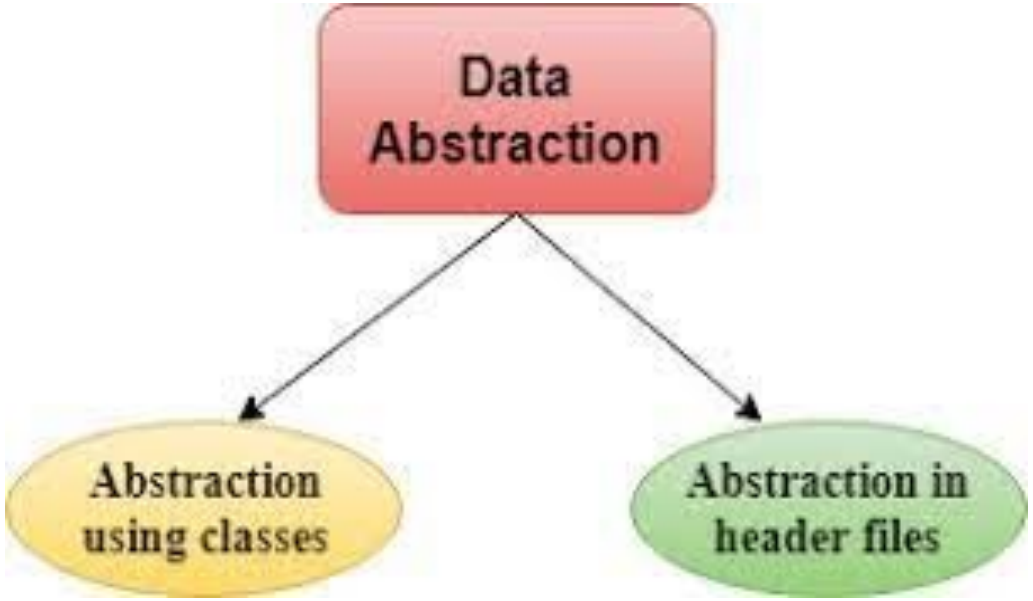
OO Paradigm


Polymorphism
Inheritance
Abstraction
OOPs Concepts
Encapsulation
Class
Object

Data Abstraction

- Abstraction refers to the act of representing essential features without including the back ground details or explanations.

- Classes use the concept of abstraction and are defined as size, width and cost and functions to operate on the attributes.



Data Encapsulation

- The wrapping up of data and function into a single unit (called class) is known as encapsulation.

- The data is not accessible to the outside world and only those functions which are wrapped in the class can access it.

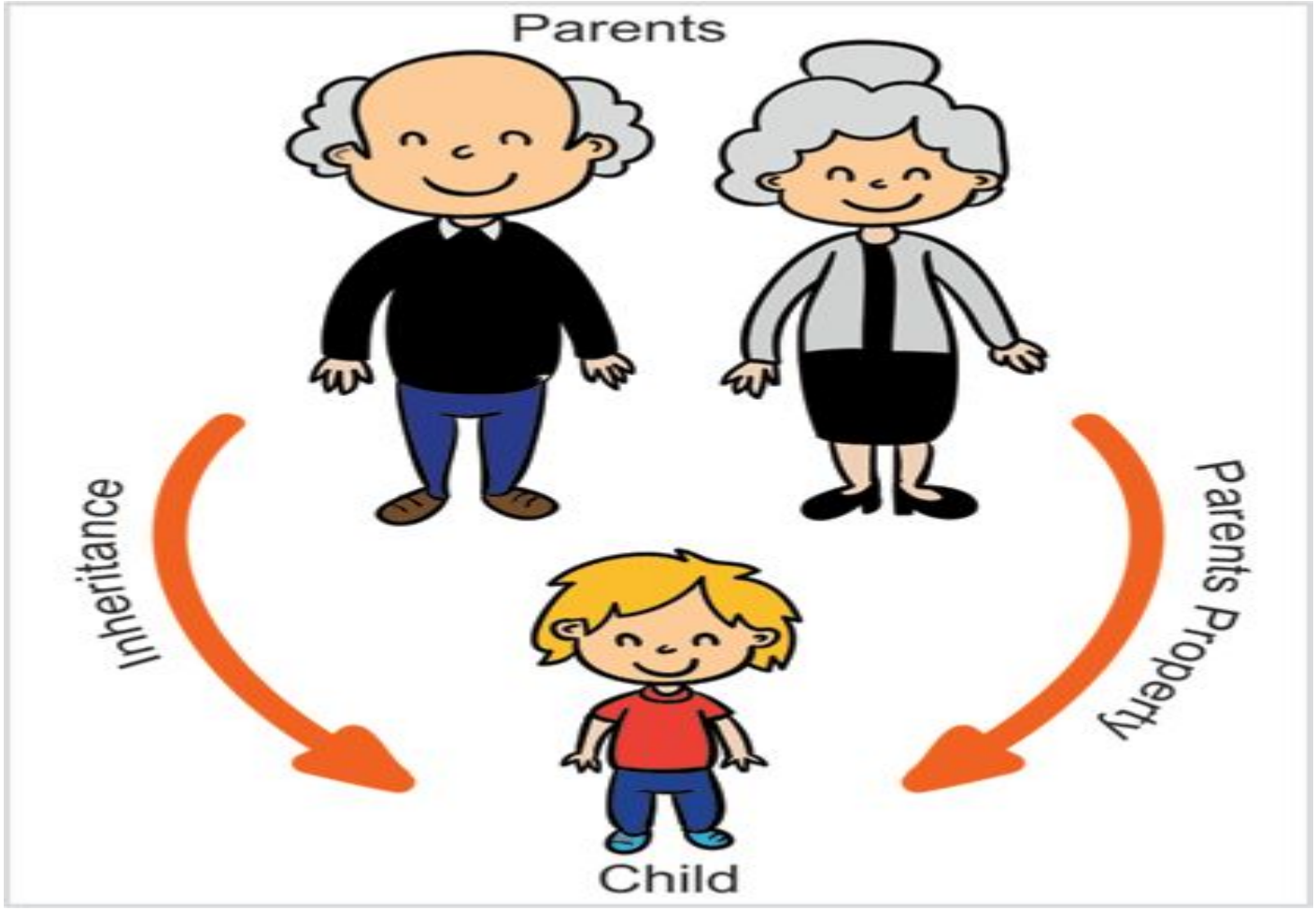
- These functions provide the interface between the objects data and the program.



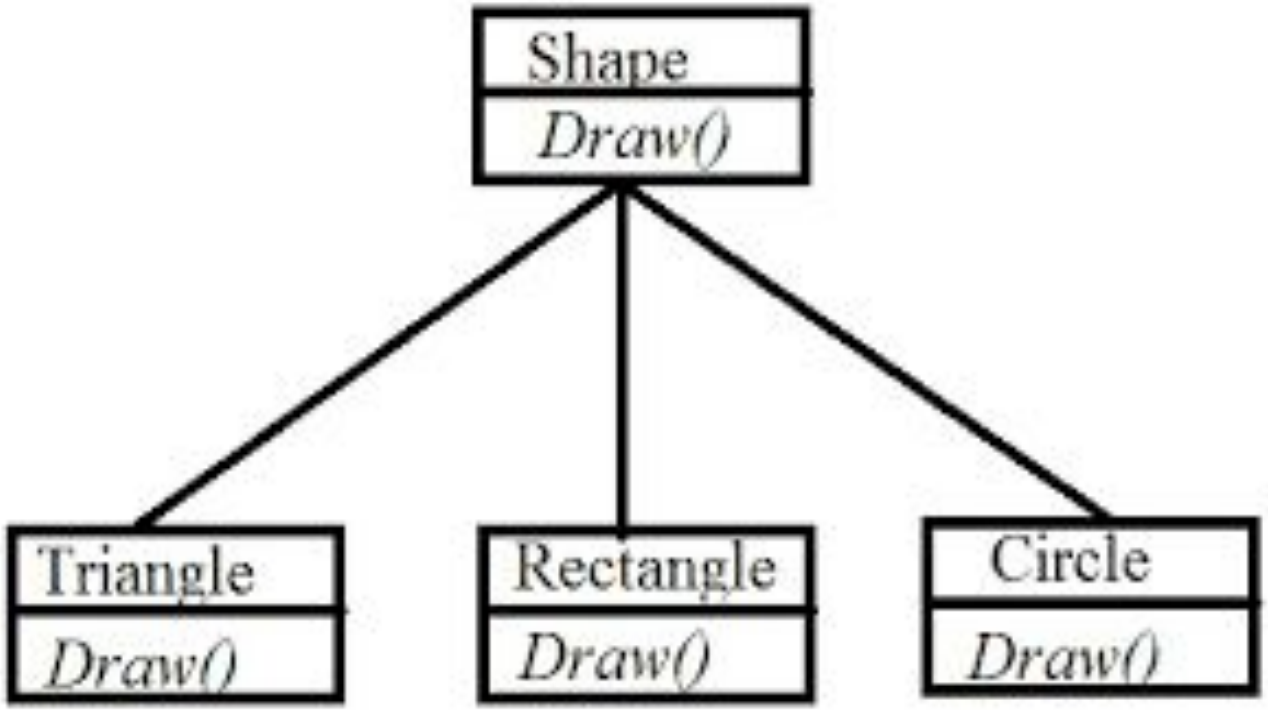
Inheritance

- Inheritance is the process by which objects of one class acquire the properties of another class.
- In the concept of inheritance provides the idea of reusablity.
- This mean that we can add additional features to an existing class with out modifying it.
- This is possible by designing a new class will have the combined features of both the classes.

Inheritance

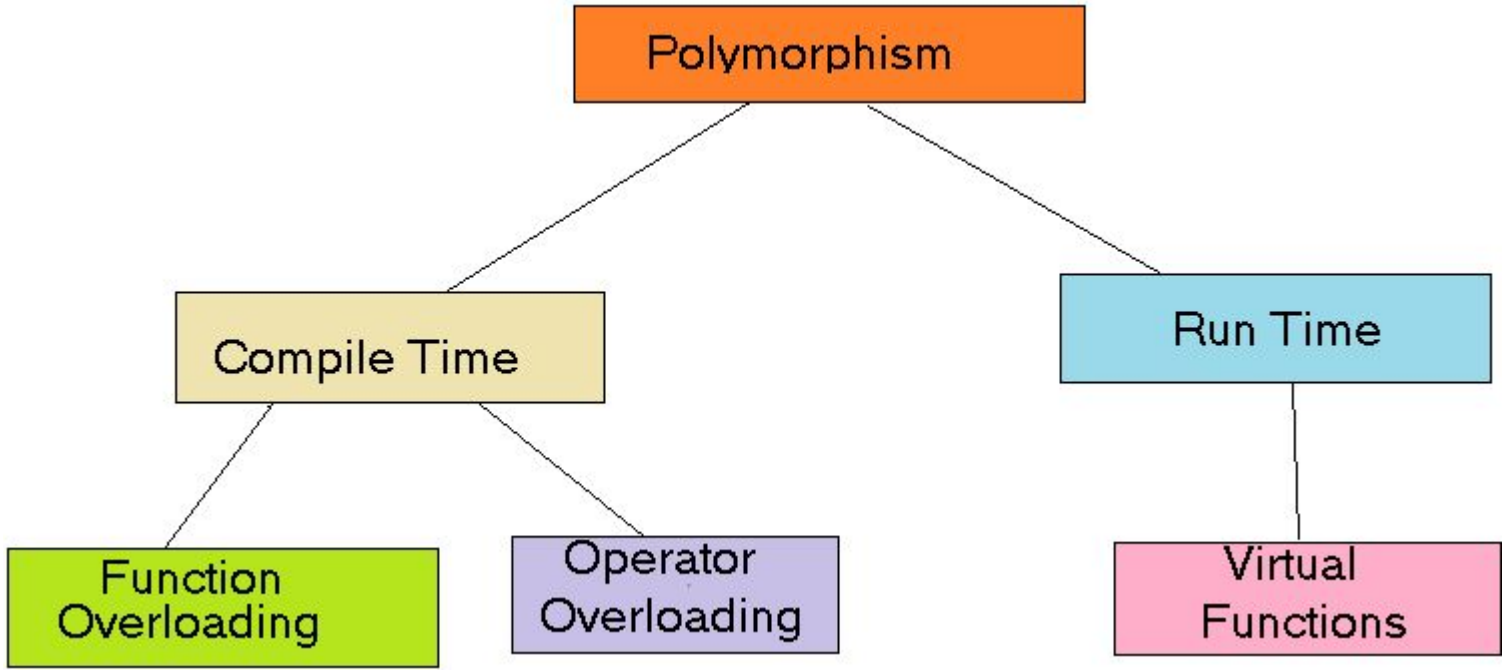


Polymorphism → One thing that can behave in multiple ways.



Polymorphism

Overloading may be operator overloading or function overloading



Others Features of OO Paradigm

- Dynamic Binding ⟶ The decision of which function to call is made during program execution, not during compilation.
- Message Passing ⟶ Communication between objects.

Methods in C++

Functions defined inside a class.

- A method is a procedure or function in OOPs Concepts.
- Whereas, a function is a group of reusable code which can be used anywhere in the program.
- This helps the need for writing the same code again and again. It helps programmers in writing modular codes.

Methods Features

- Methods have the following features:

 A method also works the same as that of function.

 A method is defined inside a class.

 A method can be private, public, or protected.

 The method is invoked by its reference/object only.

 A method is able to operate on data that is contained within the class.

 Each object has it’s own method which is present in the class

Methods Vs Function

- The main difference between methods and functions is that the methods are defined under a class, but the functions are defined outside a class i.e the functions do not belong to a particular class.
- We can call both methods and functions multiple times.
- Functions and methods do a similar job.
- They take the necessary inputs as arguments and provide us the required results.
- The main advantage of using methods is its ability to access or modify all kinds of data members of an object.
- **A method is a member function of a class, but in C++ they are more commonly called member functions than methods.**

A variable/function भी class का होता है, object का नहीं।

Static Member

- Static variables are those variables that persist even after control returns from the functions.

- Static can be either
  - Static member functions
  - Static data members.

Static Member Functions

- Static functions are functions that are made static by placing the keyword static before the function definition inside the class.
- All static functions of the class must be defined inside the class and cannot separate the declaration and definition of a static member function.
- Static functions are one for class, and you can call them as.
- In a static function only static data members, other static variables or other static function can be used.
- Though static member function is called using class name with : : operator. It can be called explicitly using objects of the class.

Example1: Demo of static function

```
#include <iostream.h>
#include <conio.h>
class demo
{
public :
static void show( )
{
cout<<“Demo of static function\n”;
}
};
```

Output

Demo of static function

Example 2: Demo of static function

```
#include <iostream.h>
#include <conio.h>
class demo
{
public :
static void show ( )
{
cout<<“Demo of static function \n”;
}
};
```

Output

Demo of static function

Demo of static function

Reason: static function are part of the class and not part

of the every object. But a static function can be called using an object too.

Static Data Member

A variable shared by all objects of the class.
There's only one copy for the entire class.

- Static data members are class members that are declared using the static keyword.
- There is only one copy of the static data member in the class, even if there are many class objects.
- This is because all the objects share the static data member.
- The static data member is always initialized to zero when the first class object is created.
- They are also known as class variables as they belong to whole of the class

Static Data Members

- An object is created separate copies of data members are created for each object. However, in case of static data members only one copy of static data members is available which is shared among all the objects created.
- Changes made by one object on a static data members is created which is shared among all objects.
- They are used when you have to keep one value common to whole class.
- The lifetime of a static variable is the entire program.

Static Data Members

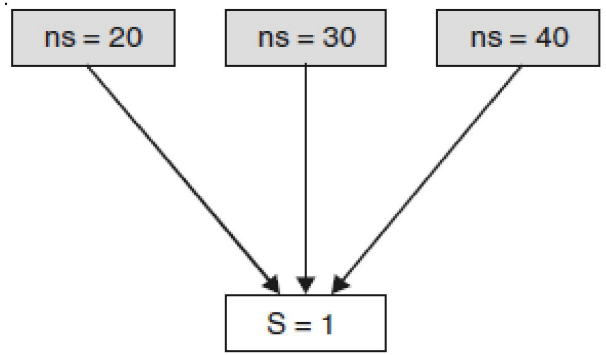
```
class demo {

static int s;

int ns;

};

demo d1, d2,d3;
```



Example: Demo of demo of static data member

```
#include <iostream>
#include<string.h>
using namespace std;
class Student
{
  private:
  int rollNo;
  char name[10];
  int marks;
  public:
  static int objectCount;
  Student()
{
    objectCount++;
  }
  void getdata() {
    cout << "Enter roll number: "<<endl;
    cin >> rollNo;
```

Example: Demo of demo of static data member

```
int Student::objectCount = 0;
int main(void) {
   Student s1;
   s1.getdata();
   s1.putdata();
   Student s2;
   s2.getdata();
   s2.putdata();
   Student s3;
   s3.getdata();
```

Output

Enter roll number: 1

Enter name: Ajay

Enter marks: 78

Roll Number = 1

Name = Ajay

Marks = 78

Enter roll number: 2

Enter name: Vijay

Enter marks: 55

Roll Number = 2

Name = Vijay

Example: Demo of demo of static data member

```
#include <iostream>
using namespace std;
class Member
{
  private:
static int A;
static int B;
static int C;
public:
static void disp ()
{
cout << " The value of the A is: " << A << endl;
cout << " The value of the B is: " << B << endl;
cout << " The value of the C is: " << C << endl;
}
};
```

Output

Print the static member through object name:

The value of the A is: 20

The value of the B is: 30

The value of the C is: 40

Print the static member through the class name:

The value of the A is: 20

The value of the B is: 30

The value of the C is: 40

Example: Demo of demo of static data member

```
#include<iostream>
using namespace std;
class number
{
static int c;
int k;
public:
void zero()
{
k=0;
}
void count()
{
++c;
++k;
cout<<"\n Value of c = "<<c;
cout<<"\n Value of k = "<<k;
}
```

Output

Value of c = 1

Value of k = 1

Value of c = 2

Value of k = 1

Value of c = 3

Value of k = 1

Example: Demo of demo of static data member

```
#include<iostream>
using namespace std;
class digit
{
static int num[10];
public :
void check(int n)
void show();
void input();
static void ini()
{
for (int k=0;k<10;k++)
num[k]=0;
}
};
void digit :: show()
{
for (int j=0;j<10;j++)
```

Example: Demo of demo of static data member

```
void digit :: check (int u)
{
int m;
while (u!=0)
{
m=u%10;
num[m]++;
u=u/10;
}
}
int main()
```

Output

Enter Numbers Number : 22151

Number 1 occurs 2 times

Number 2 occurs 2 times

Number 5 occurs 1 times

Example: Demo of demo of static data member

```
#include<iostream>
using namespace std;
class bita
{
private :
static int c;
static void count()
{
c++;
}
public:
static void display()
{
```

Output

```
Value of c : 1
Value of c : 2
```

Constructor

- A class constructor is a special member function of a class that is executed whenever we create new objects of that class.
- Constructor is used to create the object in object oriented programming language.
- A constructor will have exact same name as the class.
- It does not have any return type at all.
- Constructors can be very useful for setting initial values for certain member variables.

Characteristics of Constructors

- The constructor functions have some special characteristics which are as follows:

- They should be declared in the public section.
- They are invoked directly when an object is created.
- They don’t have return type, not even void and hence can’t return any values.
- They can’t be inherited; through a derived class, can call the base class constructor.
- Like other C++ functions, they can have default arguments.
- Constructor can be inside the class definition or outside the class definition.
- Constructor can’t be friend function.
- They make implicit calls to the operators new and delete when memory allocation is required.

Limitations of Constructor:

- C++ constructors have the following limitations:

- No return type
- Naming
- Compile time bound
- There is no virtual constructor

Declaration of Constructors

- A member function with the same name as its class is called constructor and it is used to initialize the objects of that class type with an initial value.

- Objects generally need to initialize variables or assign dynamic memory during their process of creation to become operative and to avoid returning unexpected values during their execution.

- In order to avoid that, a class can include a special function called constructor, which is automatically called whenever a new object of this class is created.

EXAMPLE

```
class Line
{
  public:
    void setLength( double len );
    double getLength( void );
    Line();

  private:
    double length;
};
```

```
Line::Line()
{
   cout << "Object is being created”;
}

void Line::setLength( double len )
{
   length = len;
}

double Line::getLength( void )
{
   return length;
}
```

```
int main( )
{
  Line line;

line.setLength(6.0);
  cout << line.getLength() <<endl;

  return 0;
}
```

Output

Object is being created

6.0

Types of Constructors

- A constructor in a class is used to initialize the value of the object.

- It prepares the new object for use by initializing its legal value.

- Constructors are classified into three main types.

- Default Constructor
- Parameterized Constructor
- Copy Constructor

1. Default Constructor

- A default constructor is a constructor that either has no parameters, or if it has parameters, all the parameters have default values.

- Having a default constructor simply means that an application can declare instances of the class.

- The compiler first implicitly defines the implicitly declared constructors of the base classes and non-static data members of a class.

- Before defining the implicitly declared constructor of A Class.

Default Constructor

```
class A {
                int i;
                public:
                void getval(void);
                void prnval(void);
                // member function definitions
          }

          A 0b 1;
// uses default constructor for creating   0b1. Since user can use it,
// that means, this implicitly defined default constructor is public
// member of the class
          0b1. Getval();
          0b1. Getval();
```

Features of default constructor

- A constructor of a class A is trivial if all the following statements are true:

- It is implicitly defined
- A has no virtual functions and no virtual base classes
- All the direct base classes of A have trivial constructors
- The classes of all the non-static data members of A have trivial constructors
- The default constructor provided by the compiler does not do anything specific.
- It simply allocates memory to data members of the object.

2. Parameterized Constructor:

- A default constructor does not have any parameter, but if you need, a constructor can have parameters.

- This helps you to assign initial value to an object at the time of its creation.

EXAMPLE 1

```
class Line
{
  public:
    void setLength( double len );
    double getLength( void );
    Line(double len);  // This is the constructor

  private:
    double length;
};

Line::Line( double len)
{
   cout << "Object is being created, length = " << len;
   length = len;
}

 void Line::setLength( double len )
{
   length = len;
}

double Line::getLength( void )
{
   return length;
}
int main( )
{
  Line line(10.0);

  // get initially set length.
  cout << "Length of line : " << line.getLength() <<endl;
  // set line length again
  line.setLength(6.0);
  cout << "Length of line : " << line.getLength() <<endl;

  return 0;
}
```

EXAMPLE 2 Multiple Constructors

```
class Line
{
  public:
    void setLength( double len );
    double getLength( void );
    Line(double len);  // This is the constructor1
    Line(double len, double wid); // This is the c2
  private:
    double length,width;
};

Line::Line( double len)
{
   cout << "Object is being created, length = " << len;
   length = len;
}
Line::Line( double len, double wid)
{
   cout << "Object is being created ";
   length = len; width= wid;
}
void Line::setLength( double len )
{
   length = len;
}

double Line::getLength( void )
{
   return length;
}

int main( )
{
  Line line(10.0);
  Line line2(20.0,20.0);
// get initially set length.
  cout << "Length of line : " << line2.getLength() <<endl;
  // set line length again
  line.setLength(6.0);
  cout << "Length of line : " << line2.getLength() <<endl;

  return 0;
}
```

Constructor with default arguments

```
class Area
{
   private:
      int length;
      int breadth;

   public:
      Area() { length=5; breadth=5; }
      void GetLength()
      {
       cout<<"Enter length and breadth respectively: ";
          cin>>length>>breadth;
      }
      int AreaCalculation() {  return (length*breadth);  }
      void DisplayArea(int temp)
      {
         cout<<"Area: "<<temp;
      }
};
```

```
int main()
{
   Area A1,A2;
   int temp;
   A1.GetLength();
   temp=A1.AreaCalculation();
   A1.DisplayArea(temp);
   cout<<endl<<"Default Area when value is not taken fro
user"<<endl;
   temp=A2.AreaCalculation();
   A2.DisplayArea(temp);
   return 0;
}
```

Constructor overloading

```
class Area
{
   private:
      int length;
      int breadth;

   public:
       Area() { length=5; breadth=5; }
      Area(int l, int b)
      {
       length =l;
       breadth =b;
       }
      void GetLength()
      {
         cout<<"Enter length and breadth : ";
         cin>>length>>breadth;
      }
      int AreaCalculation() {  return (length*breadth);  }
      void DisplayArea(int temp)
           {
              cout<<"Area: "<<temp<<endl;
           }
};
int main()
{
   Area A1,A2(2,1);
   int temp;
   cout<<"Default Area when no argument is
passed."<<endl;
   temp=A1.AreaCalculation();
   A1.DisplayArea(temp);
   cout<<"Area when (2,1) is passed as argument."<<endl;
   temp=A2.AreaCalculation();
   A2.DisplayArea(temp);
   return 0;
}
```

3. Copy Constructor

- A copy constructor is used to declare and initialize an object from another object.
- The process of initializing through a copy constructor is known as copy initialization.
- A copy constructor is always used when the compiler has to create a temporary object of a class object.

Copy Constructor

- The copy constructors are used in the following situations:

- The initialization of an object by another object of the same class.
- Return of objects as a function value.
- Stating the object as by value parameters of a function.

Copy constructor

```
#include<iostream>
class num
{
int n;
public:
num() { }
num (int k)
{
n=k;
}
num (num &j)
{
n=j.n;
}
void show (void)
 {
cout<<n;}
};
main()
{
num J(50);
num K(J);
num L=J;
num M;
M=J
cout<<"\n Object J Value of n:";

J.show();
cout<<"\n Object K Value of n:";

K.show();

cout<<"\n Object L Value of n:";

L.show();

cout<<"\n Object M Value of n:";

M.show();

return 0;
}
```

Dynamic Constructor

```
class Account
{
private:

int account_no;
int balance;

public :

Account(int a,int b)
{
account_no=a;
balance=b;
}

void display()
{
cout<< "\nAccount number is : "<< account_no;
cout<< "\nBalance is : " << balance;
}
};
```

```
void main()
{

int an, bal;

cout<< "Enter account no : ";
cin >> an;

cout<< "Enter balance : ";
cin >> bal;

Account *acc=new Account(an,bal); //dynamic
                    constructor

acc->display();

getch();
}
```

Destructors

- Destructors are functions that are complementary to constructors.
- A destructor, as the name implies, is used to destroy the objects that have been created by using the constructor.
- They de-initialize objects when they are destroyed.
- A destructor is invoked when an object of the class goes out of scope, or when the memory occupied by it is deallocated using the delete operator.

Properties of Destructor

A Destructor is different from normal functions in following ways:

- Destructor function is automatically invoked when the objects are destroyed.
- It cannot be declared static or const.
- The destructor does not have arguments.
- It has no return type not even void.
- An object of a class with a Destructor cannot become a member of the union.
- A destructor should be declared in the public section of the class.
- The programmer cannot access the address of destructor.

Example of Destructor

```
#include <iostream>
using namespace std;
class hello
{
  public:
   hello()
   {
   cout << "Constructor is Called." << endl;
   }
   ~hello()
      {
       cout << "Destructor is Called." << endl;
       }
        int display()
{
       cout << "Hello, Programmer." << endl;
   }
};
```

Output

Constructor is Called.

Hello, Programmer.

Destructor is Called.

Example of Destructor

```
#include<iostream>
using namespace std;
class Demo
{
  private:
  int num1, num2;
  void display (int, int);
  public:
  Demo(int n1, int n2) {
     cout<<"Inside Constructor"<<endl;
     num1 = n1;
     num2 = n2;
    display();
  }
~Demo() {
     cout<<"Inside Destructor";
```

Output

Inside Constructor

num1 = 10

num2 = 20

Inside Destructor

num1 = 12

num2 = 6

End of Unit 2

THANKS…..